

Data Structures

University Textbook

Table of Contents

Chapter 1 — Introduction to Data Structures

Chapter 2 — Arrays and Linked Lists

Chapter 3 — Trees and Binary Search Trees

Data Structures and Algorithms

Chapter 1 — Introduction to Data Structures

A data structure is a way of organizing and storing data in memory so that it can be accessed and modified efficiently. The choice of data structure significantly affects the performance of algorithms that operate on that data.

1.1 Classification of Data Structures

Type	Category	Examples
Primitive	Linear	int, float, char, boolean
Non-Primitive	Linear	Array, Stack, Queue, Linked List
Non-Primitive	Non-Linear	Tree, Graph, Heap
Non-Primitive	File-Based	Sequential, Indexed, Hashed

1.2 Abstract Data Types (ADT)

An Abstract Data Type (ADT) defines a data structure by its behaviour (operations) from the user's perspective, without specifying the implementation details.

Common ADTs include:

ADT	Core Operations
Stack	push(), pop(), peek(), isEmpty()
Queue	enqueue(), dequeue(), front(), isEmpty()
List	insert(), delete(), search(), traverse()
Map	put(), get(), remove(), containsKey()
Set	add(), remove(), contains(), union(), intersection()

1.3 Asymptotic Analysis

Asymptotic analysis measures the efficiency of an algorithm in terms of time and space as the input size n grows. We use asymptotic notations to express this:

- Big-O (O):** Upper bound — worst case complexity | Example: $O(n^2)$ for Bubble Sort
- Omega (Omega):** Lower bound — best case complexity | Example: $\Omega(n)$ for linear search best case
- Theta (Theta):** Tight bound — average case | Example: $\Theta(n \log n)$ for Merge Sort

1.4 Common Time Complexities

Complexity	Name	Example Algorithm
$O(1)$	Constant	Array index access
$O(\log n)$	Logarithmic	Binary Search

$O(n)$	Linear	Linear Search
$O(n \log n)$	Log-linear	Merge Sort, Heap Sort
$O(n^2)$	Quadratic	Bubble Sort, Insertion Sort
$O(2^n)$	Exponential	Recursive Fibonacci

1.5 Summary

Understanding data structures and their complexities is the foundation of algorithm design. The right data structure can reduce a solution from $O(n^2)$ to $O(n \log n)$ or better. Subsequent chapters explore each major structure — arrays, linked lists, stacks, queues, trees, and graphs — in depth.

Data Structures and Algorithms

Chapter 2 — Arrays and Linked Lists

Arrays and Linked Lists are the two most fundamental data structures. They serve as the building blocks for nearly all advanced structures including stacks, queues, hash tables, and graphs.

2.1 Arrays

An array is a contiguous block of memory storing elements of the same type, accessible via integer indices starting from 0.

Operation	Time Complexity	Notes
Access by index	O(1)	Direct memory address calculation
Search (linear)	O(n)	Must scan all elements
Search (binary)	O(log n)	Only on sorted arrays
Insert at end	O(1) amortized	May require resize
Insert at middle	O(n)	Elements must shift right
Delete at middle	O(n)	Elements must shift left

2.2 Linked Lists

A Linked List is a linear data structure where each element (node) contains a data field and a pointer (reference) to the next node in the sequence.

Types of Linked Lists:

- **Singly Linked List:** Each node points to the next node only. Traversal is unidirectional.
- **Doubly Linked List:** Each node has pointers to both next and previous nodes. Allows bidirectional traversal.
- **Circular Linked List:** The last node points back to the first node, forming a circle.

2.3 Arrays vs Linked Lists

Criterion	Array	Linked List
Memory Layout	Contiguous	Non-contiguous (scattered)
Access Time	O(1) random access	O(n) — must traverse from head
Insertion (front)	O(n) — shift needed	O(1) — update head pointer
Deletion (front)	O(n) — shift needed	O(1) — update head pointer
Memory Usage	Fixed size (static)	Dynamic — grows as needed
Cache Performance	Excellent (locality)	Poor (pointer chasing)

2.4 Implementation — Singly Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
    def insert_front(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
```

```
= Node(data) new_node.next = self.head self.head = new_node def delete_node(self, key):  
curr = self.head if curr and curr.data == key: self.head = curr.next return while  
curr.next: if curr.next.data == key: curr.next = curr.next.next return curr = curr.next
```

2.5 Summary

Arrays excel at random access and are cache-friendly, making them ideal for index-heavy operations. Linked lists excel at dynamic insertions and deletions at known positions. Choosing between them depends on the access pattern of the problem at hand.

Data Structures and Algorithms

Chapter 3 — Trees and Binary Search Trees

A tree is a hierarchical, non-linear data structure consisting of nodes connected by edges. It is widely used for representing hierarchical relationships, enabling efficient search, insertion, and deletion operations.

3.1 Tree Terminology

Term	Definition
Root	The topmost node in the tree (no parent)
Leaf	A node with no children
Height	Length of longest path from root to a leaf
Depth	Distance from the root to a given node
Subtree	A node and all its descendants
Degree	Number of children a node has
Internal Node	A node that has at least one child

3.2 Binary Tree

A binary tree is a tree in which each node has at most two children, referred to as the left child and the right child.

Types of Binary Trees:

- **Full Binary Tree:** Every node has either 0 or 2 children
- **Complete Binary Tree:** All levels are completely filled except possibly the last, which is filled from left to right
- **Perfect Binary Tree:** All internal nodes have exactly 2 children and all leaves are at the same level
- **Balanced Binary Tree:** Height of the tree is $O(\log n)$ — guarantees efficient operations
- **Degenerate Tree:** Every internal node has one child — essentially behaves like a linked list, $O(n)$ operations

3.3 Binary Search Tree (BST)

A Binary Search Tree is a binary tree with the BST property: for every node N , all keys in the left subtree are less than $N.key$, and all keys in the right subtree are greater than $N.key$.

Operation	Average Case	Worst Case (Skewed)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Min/Max	$O(\log n)$	$O(n)$

3.4 Tree Traversals

Tree traversal visits each node exactly once. The three depth-first traversal methods are:

- **Inorder (Left, Root, Right):** Visit left subtree, then root, then right subtree. Use case: Produces sorted output for BST
- **Preorder (Root, Left, Right):** Visit root, then left subtree, then right subtree. Use case: Used for tree copying and serialization
- **Postorder (Left, Right, Root):** Visit left subtree, right subtree, then root. Use case: Used for tree deletion and expression evaluation

```
def inorder(root): if root: inorder(root.left) print(root.data, end=' ')
inorder(root.right) def insert_bst(root, key): if root is None: return Node(key) if key
< root.data: root.left = insert_bst(root.left, key) else: root.right =
insert_bst(root.right, key) return root
```

3.5 Summary

Trees are fundamental to computer science. Binary Search Trees enable efficient dynamic set operations. However, in the worst case (sorted input), a BST degrades to $O(n)$. This motivates self-balancing trees such as AVL Trees and Red-Black Trees, covered in the next chapter.